From Python to MV3

A Rapid Architecture Crash Course for **Dark Pattern Guardian**

Phase 1

The Architecture Paradigm Shift

How Extensions Actually Work

1. The Webpage Context

Think of this as the user's active tab (e.g., Amazon checkout). This environment is entirely separate from your extension. It contains the DOM (Document Object Model) — the actual HTML and text we need to scan for dark patterns.

2. The Extension Context

This is where your MV3 extension lives. It operates in the background, isolated from the webpage for security. It cannot directly read the webpage text without injecting a specialized "spy" script across the boundary.

The Holy Trinity of MV3

manifest.json

The configuration file. It dictates permissions, extension name, and tells Chrome exactly which scripts to run and when.

Content Script

The DOM Extractor. This script is injected directly into the user's active tab to scrape the text payload.

Service Worker

The Background Relay. An event-driven script that listens for messages from the content script and forwards them to Flask.

Phase 2

The Configuration File

`manifest.json`: The Blueprint

- Every Chrome extension must have a root-level `manifest.json` file.
- It acts as the ID card for **Dark Pattern Guardian**, establishing its identity and capabilities.
- Because of MV3's strict privacy rules, you must explicitly declare exactly what your extension intends to do.
- If a permission isn't listed here, Chrome will block the action entirely.

Crucial Permissions for Guardian

`"activeTab"`

Allows the extension temporary access to the currently active tab when the user interacts with the extension. Critical for a privacy-first, zero-background-bloat design.

`"scripting"`

Grants the ability to inject our JavaScript (the content script) into the webpage to read the DOM.

`""` **(Host Permission)**

Allows the content script to execute on any e-commerce checkout page the user visits, ensuring universal protection.

Phase 3

Extracting the Payload

Content Scripts: The DOM Spies

- The content script (e.g., `content_bridge.js`) executes within the environment of the active webpage.
- Your primary goal here is simple: read the visible text to find fake urgency cues or hidden fees.
- In JavaScript, you access the entire visible text of a webpage using one line: `document.body.innerText`.
- This minimizes data transfer, keeping performance high—a key feature in your pitch deck.

"Content scripts live in the webpage, but they cannot talk to your local backend directly due to cross-origin security restrictions."

THE GOLDEN RULE OF MV3

Phase 4

The Background Relay

Service Workers: Ephemeral by Design

5

MINUTES MAX ACTIVE TIME

Event-Driven Execution

Unlike old MV2 scripts that ran constantly (draining RAM), an MV3 Service Worker wakes up only when an event occurs (like receiving text from your content script), processes the data, and goes back to sleep.

This is why your pitch deck promises "no persistent background bloat."

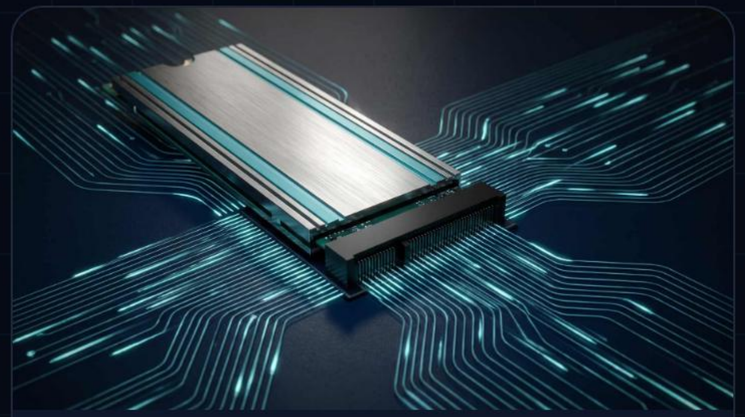
The MV3 Paradigm Shift

FEATURE	MANIFEST V2 (OLD)	MANIFEST V3 (YOUR PROJECT)
Background Architecture	Persistent Background Pages	Service Workers
Memory Footprint	High (Always running)	Low (Sleeps when idle)
DOM Access in Background	Allowed	**Strictly Prohibited**
Security & Privacy	Vulnerable to broad tracking	Highly restricted & privacy-first

Phase 5

The Communication Bridge

Message Passing Workflow



1. Extract

The content script grabs `document.body.innerText`.

2. Transmit

Uses `chrome.runtime.sendMessage` to securely cross the browser boundary.

3. Relay

The Service Worker intercepts the message and prepares to send it to Python.

Connecting to Amrutha's Backend

The `fetch()` API

Because the Service Worker acts as the central brain, it is responsible for HTTP requests. You will use JavaScript's native `fetch()` function to make a POST request containing the extracted text payload.

Localhost Routing

For the hackathon demo, the Flask backend runs locally. Your Service Worker will route the `fetch()` request to `http://127.0.0.1:5000/analyze`. Ensure the Flask server has CORS enabled to accept this request.

Your Next 3 Steps

Create the Folder Structure

Create a single folder containing exactly three files: `manifest.json`, `content_bridge.js`, and `background.js`.

Paste the Boilerplate

Copy the boilerplate code I provided earlier into those three files. Do not try to write it from scratch.

Load Unpacked & Debug

Go to `chrome://extensions`, turn on Developer Mode, and click "Load Unpacked". Open the background page console to check if text is arriving.

You're Ready.

Time to build the Guardian.

Code2Create 7.0 | Anna Auditorium | 14:00

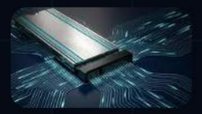
Image Sources

https://media.easy-peasy.ai/244e32a0-95a3-4888-b63b-0b90a23ce003/b899fdde-3694-43f0-9758-6389c5880225_medium.webp

Source: easy-peasy.ai

https://img.magnific.com/premium-vector/magnifying-glass-scanning-binary-code-found-petya-virus_37787-877.jpg?semt=ais_hybrid&w=740&q=80

Source: www.magnific.com



https://cdn.prod.website-files.com/698a15a8c68d8a4f171bf4b3/6a96f47773a0cc7bb08efecc_BLOG_MLPerf.jpg

Source: techarena.ai

https://images.stockcake.com/public/4/a/8/4a82d469-04e7-4572-bbb5-b4a9d2bb2137_large/glowing-network-nodes-stockcake.jpg

Source: stockcake.com

https://elements-resized.envatousercontent.com/elements-video-cover-images/files/252464891/preview.jpg?w=500&cf_fit=cover&q=85&format=auto&s=2154c8c355027f0f4d35008e98b83f04f4bccc7159ef0804ecd18891bb543f46

Source: elements.envato.com